

Subject: CS336 Lecture 2, PyTorch and Resource Accounting

Date: from 2026/7/26 to 2026/7/27

Contents

Knowledge Overview	1
Memory Accounting	1
张量与阶数	1
张量的内存	1
张量在 GPU 上	4
Compute Accounting	5
Einops	5
张量运算的 FLOPs	7
算术强度 (Arithmetic Intensity)	8
Memory and Compute Accounting for Training	11
深度网络	11
梯度基础	12
梯度的 FLOPs	13
优化器	14
训练循环	15
More Memory Optimizations	15
Summary: Resource Accounting	16

Knowledge Overview

本讲要带走的知识

- **Mechanics**（机制）. 直接明了，主要是 PyTorch 语义；
- **Mindset**（思维）. 资源核算（resource accounting）——记得去做；
- **Intuitions**（直觉）. 对资源如何被花掉形成感觉，今天没有 ML 魔法.

Memory Accounting

张量与阶数

张量：一切数据的基本单元

张量（Tensor）是用于存储一切数据的基本构建单元：

- 数据（data）；
- 参数（parameters）；
- 梯度（gradients）；
- 优化器状态（optimizer state）；
- 激活值（activations）.

阶数（rank）

每个张量都有一个阶数（rank），即其维度的数量. 下面三行分别创建 rank-1（vector）、rank-2（matrix）、rank-3 的张量：

```
x = torch.zeros(4)
x = torch.zeros(4, 8)
x = torch.zeros(4, 8, 2)
```

在 Transformer 模型中，我们会看到阶数为 4 的张量：

```
B, S, H, D = 32, 16, 16, 64
x = torch.zeros(B, S, H, D)
```

其中 B 是 batch size, S 是 sequence length, H 是 head 数, D 是每个 head 的 hidden dimension.

张量的内存

内存由元素数量与数据类型决定

张量的元素通常是浮点数. 内存由 (i) 值的数量和 (ii) 每个值的数据类型共同决定. 下面的断言说明：默认类型是 fp32，元素数为 4×8 ，每个浮点数占 4 字节，因此共 128 字节.

```
x = torch.zeros(4, 8)
assert x.dtype == torch.float32
assert x.numel() == 4 * 8
assert x.element_size() == 4
assert get_memory_usage(x) == 4 * 8 * 4
```

GPT-3 前馈层中的一个矩阵:

```
assert get_memory_usage(
    torch.empty(12288 * 4, 12288)) == 2304 * 1024 * 1024
```

也就是约 2.3 GB.

fp32 (单精度)

fp32 (也叫 float32 或单精度) 是默认选项. 传统上在科学计算中 fp32 是基准, 某些情况下还会用双精度 fp64. 在深度学习中, 你可以“马虎”很多.

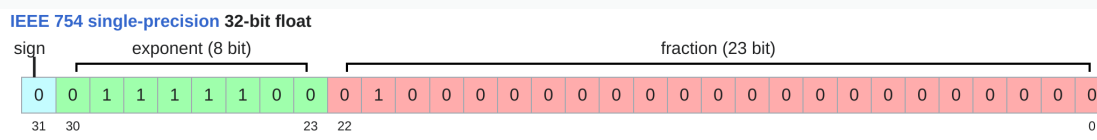


Figure 1: IEEE 754 单精度 32 位浮点的位布局.

fp16 (半精度)

fp16 (也叫 float16 或半精度) 可以减少内存使用, 每个元素只占 2 字节.

```
x = torch.zeros(4, 8, dtype=torch.float16)
assert x.element_size() == 2
```

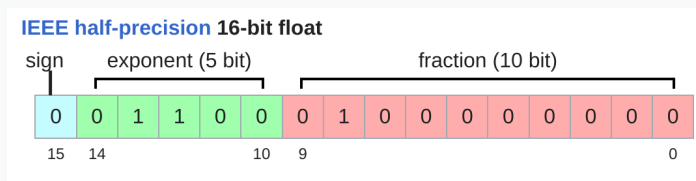


Figure 2: fp16 的位布局.

然而它的动态范围表现不佳, 尤其是对小数字, 会发生下溢 (Underflow):

```
x = torch.tensor([1e-8], dtype=torch.float16)
assert x == 0
```

如果在训练时发生这种情况, 可能导致训练不稳定.

bf16（脑浮点）

Google Brain 在 2018 年开发了 bfloat16（brain floating point）来解决上述问题。

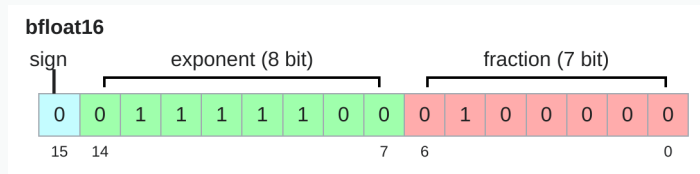


Figure 3: bf16 与 fp32 共享指数位，因此动态范围相同。

bf16 使用与 fp16 相同的内存，但拥有与 fp32 相同的动态范围。唯一的缺点是精度（分辨率）较低，但这在深度学习中影响不大。

```
x = torch.tensor([1e-8], dtype=torch.bfloat16)
assert x != 0
```

混合精度训练（Mixed precision）

对训练的影响：

- 用 fp32 训练可行，但需要大量内存；
- 用 fp16 甚至 bf16 训练存在风险，可能导致训练不稳定。

解决方案：混合精度训练（Micikevicius+ 2017）。

- 参数、激活值和梯度用 bf16；
- 优化器状态用 fp32。

PyTorch 有自动混合精度（AMP）库，它会尝试在安全的情况下（例如矩阵乘法 `matmul`，而非 `exp` 操作）把计算转换为 bf16。

```
with torch.amp.autocast("cuda", dtype=torch.bfloat16):
    x = torch.zeros(4, 8)
```

fp8

2022 年，受机器学习工作负载的推动，fp8 被标准化。

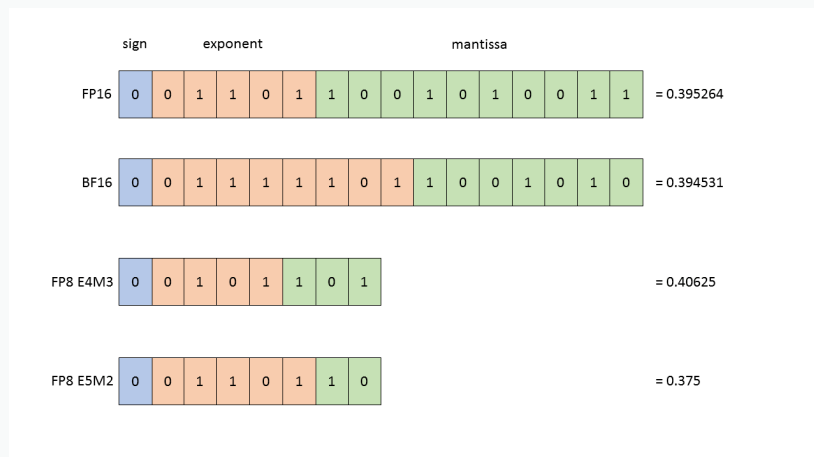


Figure 4: fp8 的两种格式 E4M3 与 E5M2.

- H100 GPU 支持两种 fp8 变体: **E4M3** (范围 $[-448, 448]$) 与 **E5M2** (范围 $[-57344, 57344]$);
- 参考: Micikevicius+ 2022.

fp4 (nvfp4)

2025 年, NVIDIA 开发了 nvfp4.

- 每个值仅占 4 位;
- 可表示的数值为 -6, -4, -3, -2, -1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 1.5, 2, 3, 4, 6;
- 每个块 (block) 使用单独的缩放因子, 因此实际上可以获得更大的动态范围 (只是不能与邻近值自由变化);
- Nemotron 3 Super 模型正是在 NVFP4 精度下训练的;
- 其中部分操作是在用户控制之外的 NVIDIA 库中完成的.

张量在 GPU 上

CPU 与 GPU 内存

默认情况下, 张量存储在 CPU 内存中:

```
x = torch.zeros(32, 32)
assert x.device == torch.device("cpu")
```

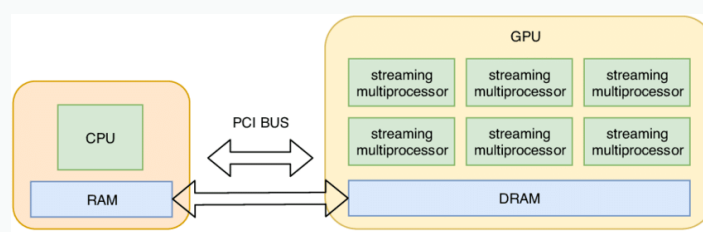


Figure 5: CPU 内存与 GPU 内存之间的 PCIe 传输.

为了利用 GPU 的大规模并行计算能力，需要把张量移动到 GPU 内存中（`cuda_if_available()` 在可用时返回“cuda”，否则返回“cpu”）：

```
device = cuda_if_available()
x = x.to(device)
```

或者直接在 GPU 上创建张量：

```
with torch.device(device):
    x = torch.zeros(32, 32)
assert x.device == device
```

Compute Accounting

Einops

Einops 的动机

Einops 是一个用于操作张量的库，其特点是**维度被命名**。它的灵感来自爱因斯坦求和约定（Einstein, 1916）。

einsum：广义矩阵乘法

Einsum 是一种广义矩阵乘法，具有良好的“记账”能力（能清晰地追踪维度）。

```
x = torch.ones(3, 4)
y = torch.ones(4, 3)
z = x @ y
```

用 einops 写就是：

```
z = einsum(x, y, "seq1 hidden, hidden seq2 -> seq1 seq2")
```

再看一个更复杂的例子，其中 batch 维度需要广播：

```
x = torch.ones(2, 3, 4)
y = torch.ones(2, 3, 4)
z = x @ y.transpose(-2, -1)
```

用 einops 写就是：

```
z = einsum(x, y,
           "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
```

- 在输出中没有指定的维度会被求和（约减掉）；
- 用... 表示对任意数量维度的广播；

```
z = einsum(x, y, "... seq1 hidden, ... seq2 hidden -> ... seq1 seq2")
```

reduce: 约减单个张量

可以通过某种操作（求和、平均、最大、最小）来约减（reduce）单个张量。设

```
x = torch.ones(2, 3, 4)
```

也就是形状为 (batch, seq, hidden). 目标是：对最后一个维度（hidden，大小为 4）求和，得到形状 (2, 3) 的张量。

旧方法（PyTorch 原生）。

```
y = x.sum(dim=-1)
```

dim=-1 指最后一个维度，即 hidden：把大小为 4 的那个维度全部加起来，得到一个数字取代原来的 4 个位置。结果形状从 (2, 3, 4) 变成 (2, 3)。这个过程就叫“约减（reduce）”，因为我们在某个维度上做了归并，减少了张量的总元素数量。

新方法（einops）。

```
y = reduce(x, "... hidden -> ...", "sum")
```

对于张量中所有前面的维度 (...), 取其中的 hidden 维度，用 sum 把它约减掉，最终只保留前面的维度。

rearrange: 拆分与合并维度

有时一个维度实际上代表了两个维度（例如被压平在一起），而你想对其中一个维度进行操作。下面用二维张量模拟“多头”线性变换。

- 输入：x 形状 (3, 8)，即 3 个样本（seq），每个的特征维度是 8（total_hidden）；
- 权重：w 形状 (4, 4)；
- 目标：把 8 维特征拆成 2 个头，每个头 4 维（hidden1），用 w 把这 4 维变成新的 4 维（hidden2），最后再合并回 8 维。

Step 1: rearrange —— 拆分维度

```
x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)
```

括号 () 表示把一个维度拆成两个：原来的 total_hidden=8 被拆成 heads=2 与 hidden1=4。形状从 (3, 8) 变成 (3, 2, 4)。

Step 2: einsum —— 组内变换

```
x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
```

此时 x 形状是 (3, 2, 4) (seq, heads, hidden1)，w 形状是 (4, 4)。对每个 seq 和每个 heads，用那组的 4 维向量去和 w 做矩阵乘法。形状不变（输入 4 维、输出也是 4 维），但数据内容

发生了变化.

Step 3: rearrange ——合并维度

```
x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")
```

这是第一步的逆过程：把 heads 与 hidden2 重新压平合并. 形状从 (3, 2, 4) 变回 (3, 8).

张量运算的 FLOPs

FLOPs 与 FLOP/s

浮点运算 (FLOP) 是一种基本运算, 如加法 $x + y$ 或乘法 $x * y$. 有两个发音相同、极易混淆的缩写:

- **FLOPs**: 浮点运算次数 (衡量完成的计算量);
- **FLOP/s**: 每秒浮点运算次数 (也写作 FLOPS), 衡量硬件的速度.

一些直观数据

- 训练 GPT-3 (2020 年) 花费了 3.14×10^{23} FLOPs;
- 训练 GPT-4 (2023 年) 据推测花费了 2×10^{25} FLOPs;
- H100 GPU 的峰值性能: 开启稀疏性时 1979 万亿次 FLOP/s, 不开启时约减半 (约 990 万亿次 FLOP/s).

```
h100_flop_per_sec = 1979e12 / 2
```

8 块 H100 运行 2 周:

```
total_flops = 8 * 2 * (60 * 60 * 24 * 7) * h100_flop_per_sec
```

线性模型示例

设 B 为点的数量 (批量大小) = 16384, D 为每个点的维度 = 32768, K 为输出数量 = 8192.

```
B = 16384
D = 32768
K = 8192
```

```
x = torch.ones(B, D, device=cuda_if_available())
w = torch.randn(D, K, device=cuda_if_available())
y = x @ w
```

这个矩阵乘法需要多少次 FLOPs? 对每个 (i, j, k) 三元组, 我们有一次乘法和一次加法, 因此

```
actual_num_flops = 2 * B * D * K
```

也可以计时，看它需要多长时间，从而算出实际 FLOP/s:

```
actual_time = benchmark(lambda: x @ w)
actual_flop_per_sec = actual_num_flops / actual_time
```

每块 GPU 都有规格表给出峰值性能. 注意，FLOP/s 高度依赖于数据类型:

```
promised_flop_per_sec = get_promised_flop_per_sec(x.dtype)
```

MFU (Model FLOPs Utilization)

定义: $\text{MFU} = \text{实际 FLOP/s} \div \text{承诺 / 峰值 FLOP/s}$ (忽略通信与开销).

```
mfu = actual_flop_per_sec / promised_flop_per_sec
```

通常, $\text{MFU} \geq 0.5$ 就算相当不错了! 为什么它不能更接近 1 呢? 要回答这个问题, 需要更仔细地研究计算在 GPU 上是如何执行的.

算术强度 (Arithmetic Intensity)

一次计算如何执行

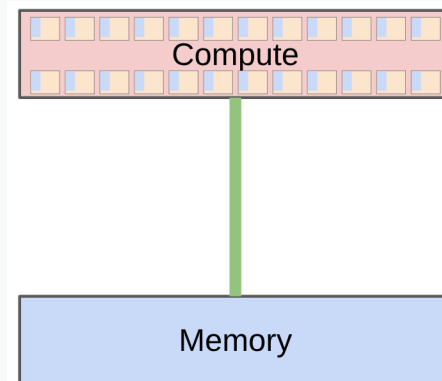


Figure 6: 计算单元与内存之间的数据搬运.

1. 将输入从内存发送到加速器 (如 GPU);
2. 执行计算;
3. 将输出从加速器发送回内存.

这需要多长时间? 取决于两件事:

- 加速器速度 (FLOP/s): 每秒能执行多少次浮点运算;
- 内存带宽 (bytes/s): 每秒能传输多少字节的数据.

下面的第二行就是 H100 的内存带宽:

```
assert h100_flop_per_sec == 1979e12 / 2
assert h100_bytes_per_sec == 3.35e12
```

假设通信和计算可以完美地重叠，则

```
total_time = max(communication_time, computation_time)
```

加速器强度与瓶颈判定

- **加速器强度**. 硬件分界线，表示 GPU 每从显存搬 1 byte 数据，至少要做多少 FLOPs 才能把计算单元也忙满：

```
h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
```

- **算术强度**. 对于某个任务，每传输 1 字节实际完成了多少计算：

```
arithmetic_intensity = flops / bytes
```

瓶颈判定：

- **内存受限 (Memory-bound)**: 算术强度 < 加速器强度，即通信时间 > 计算时间；
 - **计算受限 (Compute-bound)**: 算术强度 > 加速器强度，即计算时间 > 通信时间.
- 一般来说，我们会发现自己常常处于**内存受限**的状态.

ReLU

下面的 bytes 是读 x、写 y 的字节数 (bfloat16 每个数 2 字节)，flops 是 n 次比较操作.

```
n = 1024 * 1024
x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
y = torch.relu(x)

bytes = (2 * n) + (2 * n)
flops = n
arithmetic_intensity = flops / bytes
assert arithmetic_intensity < h100_accelerator_intensity
```

结论: **ReLU** 是内存受限的.

GELU

这里的 flops 取 $20n$ ，因为 tanh 可以用多项式等方式近似.

```

y = F.gelu(x)
bytes = (2 * n) + (2 * n)
flops = 20 * n
arithmetic_intensity = flops / bytes
assert arithmetic_intensity < h100_accelerator_intensity

```

GELU 每传输 1 字节做了更多的计算，因此算术强度高于 ReLU. 但它仍然是内存受限的. 换句话说，**ReLU 并不比 GELU 更快**（在孤立地单独执行时）.

点积

```

y = x @ w
bytes = (2 * n) + (2 * n) + 2
flops = 2 * n - 1
arithmetic_intensity = flops / bytes
assert arithmetic_intensity < h100_accelerator_intensity

```

其中 flops 是 n 次乘法加 $n - 1$ 次加法，算术强度约为 $1/2$ ，属于内存受限.

矩阵-向量乘法

```

y = x @ w
bytes = (2 * n) + (2 * n * n) + (2 * n)
flops = n * (2 * n - 1)
arithmetic_intensity = flops / bytes
assert arithmetic_intensity < h100_accelerator_intensity

```

其中 flops 对应 n 个点积，算术强度约为 1，属于内存受限.

矩阵-矩阵乘法

```

y = x @ w
bytes = (2 * n * n) + (2 * n * n) + (2 * n * n)
flops = n * n * (2 * n - 1)
arithmetic_intensity = flops / bytes
assert arithmetic_intensity > h100_accelerator_intensity

```

其中 flops 对应 n^2 个点积，算术强度约为 $n/3$ ，属于**计算受限**.

- 只要矩阵足够大，我们就是计算受限的（能用满加速器的计算能力）；
- 训练 Transformer 涉及大规模矩阵乘法，因此是计算受限；
- 矩阵-向量乘法出现在**推理（inference）**阶段，这就是推理是内存受限的原因；
- 注意：算术 / 加速器强度也取决于**精度**（bf16 与 fp32 会有差异）.

屋顶线图（Roofline plots）

可以用屋顶线图来可视化算术强度与性能之间的关系。

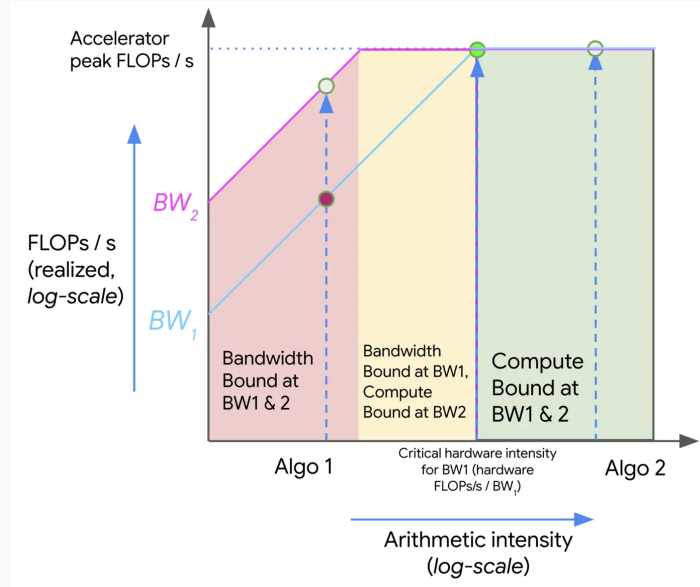


Figure 7: 屋顶线图：不同带宽下的内存受限区与计算受限区。

- x 轴上的每个切片代表一个特定的计算操作（具有某个算术强度值）；
- 每个分段线性函数对应一种特定的硬件；
- 拐点（kink）就是加速器强度，即从内存受限过渡到计算受限的分界点。

这样就能把屋顶线图与 MFU 联系起来：

$$MFU = \min\left(1, \frac{\text{算术强度}}{\text{加速器强度}}\right)$$

Memory and Compute Accounting for Training

深度网络

深度网络：L 层、D 维

考虑一个深度网络，它有 L 层，且输入、激活值和输出都是 D 维的。

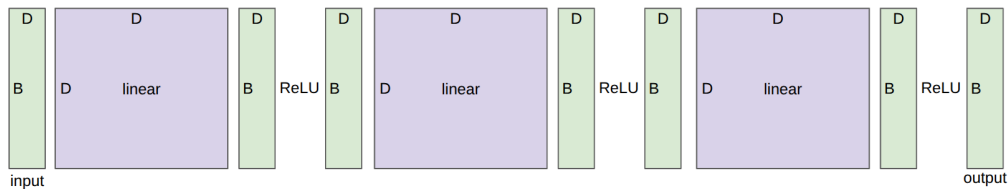


Figure 8: 深度网络：每层是线性变换加 ReLU。

下面 D 是输入 / 激活值 / 输出的维度（8），L 是层数（3），B 是批量大小（4）。每层有 $D \times D$ 个参数，因此参数数量为 D^2L ：

```

D = 8
L = 3
model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())

num_parameters = get_num_parameters(model)
assert num_parameters == (D * D) * L

B = 4
x = torch.randn(B, D, device=cuda_if_available())
y = model(x)

```

网络定义

```

class Block(nn.Module):
    def __init__(self, dim: int):
        super().__init__()
        self.weight = nn.Parameter(
            torch.randn(dim, dim) / math.sqrt(dim))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x @ self.weight
        x = F.relu(x)
        return x

class DeepNetwork(nn.Module):
    def __init__(self, dim: int, num_layers: int):
        super().__init__()
        self.layers = nn.ModuleList(
            [Block(dim) for i in range(num_layers)])

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        for layer in self.layers:
            x = layer(x)
        return x

```

梯度基础

前向传播与反向传播

作为一个简单的例子，考虑线性模型

$$y = 0.5 (x \cdot w - 5)^2$$

前向传播：计算损失

```
x = torch.tensor([1., 2, 3])
w = torch.tensor([1., 1, 1], requires_grad=True)
pred_y = x @ w
loss = 0.5 * (pred_y - 5).pow(2)
```

反向传播：计算梯度

```
loss.backward()
assert torch.equal(w.grad, torch.tensor([1, 2, 3]))
```

梯度的 FLOPs

loss.backward() 做了什么

loss.backward() 是 PyTorch 自动求导机制的“总开关”，它只做一件核心的事：计算所有 requires_grad=True 的张量的梯度。它本身不返回任何值，而是把计算结果“存”在每个张量的.grad 属性里。

调用后可以直接访问 w1.grad、w2.grad（形状与参数相同）。为了算出它们，反向传播内部实际上产生了一条反向流水线：

1. **起点**. 从 loss 开始，梯度初始为 1.0；
2. **第二层**. 反向到 h2，得到 h2.grad；它是临时的，需用 h2.retain_grad() 强制保留，否则会被释放；
3. **第三层**. 利用 h2.grad 与 w2，同时算出 w2.grad 与 h1.grad（传给下一层的中间结果）；
4. **继续反向**. 利用 h1.grad 与 w1，算出 w1.grad.

因此，反向传播内部“得到”了整条链上的所有中间梯度，但只有那些 requires_grad=True 的张量的梯度才会被保存下来。其他中间结果默认被丢弃以节省内存，除非用 retain_grad() 明确要求保留。

聚焦一层：前向与反向

下面设 B 为数据点数量（1024），D 为维度（256）。

```

B = 1024
D = 256

x = torch.ones(B, D, device=cuda_if_available())
w1 = torch.randn(D, D, device=cuda_if_available(), requires_grad=True)
w2 = torch.randn(D, D, device=cuda_if_available(), requires_grad=True)

h1 = einsum(x, w1, "batch in, in out -> batch out")
h2 = einsum(h1, w2, "batch in, in out -> batch out")
loss = (h2.mean() - 0)**2

h1.retain_grad()
h2.retain_grad()
loss.backward()

```

聚焦第二层 ($h2 = h1 @ w2$):

- 前向传播: $\text{num_forward_flops} = 2 * B * D * D$;
- 反向传播: 需要计算 $h1.grad$ 与 $w2.grad$.

```

h1_grad = einsum(h2.grad, w2, "batch out, in out -> batch in")
assert torch.allclose(h1.grad, h1_grad)

w2_grad = einsum(h2.grad, h1, "batch out, batch in -> in out")
assert torch.allclose(w2.grad, w2_grad)

```

这两次运算的 FLOPs 为 $\text{num_backward_flops} = (2*B*D*D) + (2*B*D*D)$. 注意, 反向传播的计算量是前向传播的 2 倍.

汇总: $6 \times \text{数据点} \times \text{参数量}$

把网络中所有层的参数都算上, 汇总得到:

- 前向传播: $2 \times (\text{数据点数量}) \times (\text{参数量})$ FLOPs;
- 反向传播: $4 \times (\text{数据点数量}) \times (\text{参数量})$ FLOPs;
- 总计: $6 \times (\text{数据点数量}) \times (\text{参数量})$ FLOPs.

这个计算适用于多层感知机 (MLP). 事实证明, 对于短上下文长度的 Transformer 模型, 这个近似值也同样适用.

优化器

优化器谱系

- **Momentum** = SGD + 梯度的指数移动平均;
- **AdaGrad** = SGD + 按梯度平方累积进行缩放;
- **RMSProp** = AdaGrad + 梯度平方的指数移动平均;
- **Adam** = RMSProp + Momentum.

AdaGrad 与一步优化

```
optimizer = AdaGrad(model.parameters(), lr=0.01)
state = model.state_dict()
```

计算梯度：

```
x = torch.randn(B, D, device=cuda_if_available())
y = torch.tensor([4., 5.], device=cuda_if_available())
pred_y = model(x).mean()
loss = F.mse_loss(input=pred_y, target=y)
loss.backward()
```

执行一步优化，然后释放内存：

```
optimizer.step()
optimizer_state = {i: dict(p_state)
                   for i, (p, p_state) in
                   enumerate(optimizer.state.items())}

optimizer.zero_grad(set_to_none=True)
```

训练循环

train_loop()

一次完整的训练迭代包含：清零梯度、前向传播、计算损失、反向传播、更新参数。

```
for x, y in data_loader:
    pred_y = model(x)
    loss = F.cross_entropy(pred_y, y)

    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
```

把 zero_grad 放在前向之前或之后都可以，关键是在下一次反向传播累积梯度之前清零。

More Memory Optimizations

梯度累积 (Gradient accumulation)

当显存装不下目标 batch size 时，可以把一个大 batch 拆成多个 micro-batch：分别做前向与反向，**累积梯度**，累积若干步后再更新一次参数。这样就以有限的显存模拟出更大的 batch size。

注意：为了让累积后的梯度等价于一个大 batch，通常要把每个 micro-batch 的 loss 除以累积步数 accum_steps。

```

for i, (x, y) in enumerate(micro_batches):
    loss = compute_loss(model(x), y) / accum_steps
    loss.backward()

    if (i + 1) % accum_steps == 0:
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)

```

激活检查点 (Activation checkpointing)

反向传播需要前向计算的中间**激活值**，它们会占用大量显存。激活检查点用**计算换内存**：只保存少量检查点，反向传播时重新计算需要的中间激活。

- **收益**. 显著降低激活值占用的显存，从而支持更大的 batch 或更深的模型；
- **代价**. 额外的前向计算，通常会让单步训练变慢。

Summary: Resource Accounting

本讲小结：记住这几个数

- **显存**. 张量占用 = 元素数量 $\times$ 每个元素的字节数 (fp32 = 4, bf16 / fp16 = 2, fp8 = 1)；
- **训练计算量**. 约 $6 \times (\text{数据点}) \times (\text{参数量})$ FLOPs: 其中前向 $2ND$ ，反向 $4ND$ ；
- **FLOPs 与 FLOP/s**. 前者是计算量，后者是硬件速度，不要混淆；
- **算术强度**. flops/bytes; 加速器强度 = 峰值 FLOP/s $\div$ 内存带宽；
- **瓶颈判定**. 算术强度 $<$ 加速器强度 $\Rightarrow$ 内存受限，否则为计算受限；
- **训练与推理**. 训练以大规模 matmul 为主，是计算受限；推理（矩阵-向量）是内存受限；
- **MFU**. $\min(1, \text{算术强度}/\text{加速器强度})$ ，通常 ≥ 0.5 已相当不错。